- Compute Match — System Architecture
  - Runtime components
  - Backend architecture (DDD + hexagonal)
  - Division of labour between backend and worker
  - Data stores
  - Frontend architecture
  - Cloud platform

Compute Match — System Architecture

Runtime components

```
Customer / NCP / Admin browser
          |
   Regional Gateway (GCP load balancer + Cloud Armor)
     ├── Frontend (Next.js)  ──▶ Backend API, Keycloak
     ├── Backend API (FastAPI) ──▶ Cloud SQL PostgreSQL, Memorystore Redis, Temporal Cloud,
Keycloak, Salesforce, email
     ├── Keycloak (identity, Starfleet SSO broker) ──▶ PostgreSQL
     └── Superset (analytics) ──▶ PostgreSQL (analytics schema)
Workflow worker (Temporal) ──▶ Backend System APIs, PostgreSQL, Temporal
```

Component	Source folder	Responsibility
Backend	`backend/`	REST API, domain logic, migrations, email, persistence, integrations
Frontend	`frontend/`	Browser application and authentication client
Shared workflows	`shared/workflows/`	Temporal workflow and activity definitions

Component	Source folder	Responsibility
Workflow worker	`workflow_worker/`	Runs Temporal workflows, activities and schedules
Keycloak	`keycloak/`, `keycloak-extensions/`	Identity provider, themes, realm config, custom Java extensions
Superset	`superset/`	Dashboards and reports
E2E tests	`e2e_playwright/`	Playwright + Cucumber browser tests
Local setup	`local-setup/`	Docker Compose development stack
Infrastructure	`infrastructure/`	Terraform, Helm charts, environment values
Tools	`tools/`	Maintenance and developer helpers
Python SDK	`backend-sdk-python/`	Generated OpenAPI client used by the worker

Backend architecture (DDD + hexagonal)

- `backend/core/` — the domain. Bounded contexts: `gpu_offering`, `reservation_request`, `hosting_provider`, `geographical_region`, `gpu_model`, `organization`, `user`, `allocation`, `access_request`, `provider_onboarding`, `feedback`, `audit`, `authorization`, `partner_domain`, `salesforce`. Ports (interfaces) and domain services live here; `kernels/` holds shared ids, enums and errors.
- `backend/infrastructure/driving/http/` — the driving adapter: FastAPI app factory (`main.py`), routers, auth middleware, request schemas, rate limiting.
- `backend/infrastructure/driven/` — driven adapters: SQLAlchemy database models and repositories with Alembic migrations, email, Keycloak admin, Temporal clients, Salesforce, feedback.
- The domain never imports infrastructure; adapters implement the core ports.

Division of labour between backend and worker

- The **backend owns all domain changes** (status transitions, imports, emails rendering).

- The **worker owns time**: timers, schedules, retries, fan-out. Its activities call back into backend System APIs using a Keycloak service account (`workflow-worker` client) with scopes `notification:send`, `capacity:sync`, `reservation-request:write`.

Data stores

- **PostgreSQL** (Cloud SQL in GCP) — application data, Keycloak data, analytics views (`analytics` schema for Superset), audit log (purged after about 13 months).
- **Redis** (Memorystore) — rate limiting buckets (default, mutate, auth, import, availability, feedback), optional cache, role-sync budgets.
- **Temporal** — workflow state (Temporal Cloud in shared environments, local Temporal in Docker for development).

Frontend architecture

- Next.js 15 App Router (`frontend/src/app/**/page.tsx`), React 19, Ant Design 6, Tailwind CSS 4, Roboto font.
- Authentication with NextAuth 5 + Keycloak; middleware (`frontend/src/middleware.ts`, `frontend/src/services/auth/config.ts`) protects `/customer` and `/provider` routes and enforces status gates and scopes.
- State: Zustand stores (marketplace, inventory, What's New) and SWR for server data; Axios for backend calls (`frontend/src/services/backend/*.ts`).
- Maps and charts with D3 and topojson; react-markdown for the in-app User Manual (`frontend/src/content/USER_MANUAL.md`).
- What's New content lives in `frontend/src/content/whats-new.json`.
- UI rules: `frontend/UI_STANDARDS.md` (page headers, filters, chips, Title Case buttons, WCAG 2.2 AA).

Cloud platform

Private **GKE Autopilot** clusters on Google Cloud, Cloud SQL PostgreSQL, Memorystore Redis, Secret Manager, regional Gateway load balancing, Cloud Armor, network policies. Temporal Cloud for orchestration. Terraform owns cloud resources; Helm owns workloads; GitLab CI builds and promotes immutable images.